REPORT

Reviewer report: urllib3/urllib3#5368: Fix

ProxySchemeUnknown message and scheme on Python

3.14

Run urllib3-5368, tested 2026-11-09 11:50 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 18:39 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Fix ProxySchemeUnknown message and scheme on Python 3.14 (urllib3/urllib3#5368).

• What we observed: probe: 1 observation(s) need attention: c4.has_scheme_none differs although the claim

says it does not (C4); probe: 2 observation(s) shown as advisory only (claim not shown on these inputs; a related

change was proven): c1.message (C1); c8.message_matches (C8); probe: claim(s) C2, C3, C5, C7, C9 proven;

tests: 20 test(s) ran on main, 23 on the PR; 3 new test(s) pass, 20 unchanged.

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 5.9 GB (of 15.6 GB).

• Setup check: free disk 92.9 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: urllib3 python setup install instructions https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python documentation install https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python install requirements https://pypi.org/project/urllib3/ https://github.com/urllib3/urllib3

site:pypi.org

• Source: urllib3 · PyPI — https://pypi.org/project/urllib3/

• Source: urllib3 installation requires setuptools >= 36.2.5 · Issue #1753 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1753

• Source: urllib3/docs/user-guide.rst at main · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/blob/main/docs/user-guide.rst

• Source: ImportError during install of urllib3[secure] on Jython · Issue #1133 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1133

• Source: GitHub - urllib3/urllib3: urllib3 is a user-friendly HTTP client library for Python · GitHub —

https://github.com/urllib3/urllib3

• Source: Installing urllib3 · Issue #829 · urllib3/urllib3 · GitHub — https://github.com/urllib3/urllib3/issues/829

• Source: No module named 'urllib3.packages.six' · Issue #1518 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1518

• Setup plan: the same packages on main and pr.

• Setup plan: install trustme>=1.2.1 (listed in the test extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10

• Installed trustme 1.2.1 from pyproject (listed in the test extra) on main and pr.

Setup: pytest plugin asyncio was turned off. Pytest stopped at startup the same way on both sides, and the

traceback pointed into that third-party plugin (AttributeError: module 'pytest' has no attribute

'pyodide_options_stack'). The tests were run again with the plugin turned off. This does not change how the

verdict is chosen. Detection fired. Pytest stopped at startup the same way on both sides. Plugin chosen: asyncio.

The installed pytest plugin list was read. Retry flags: -p no:asyncio. Outcome: worked.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (20 unchanged, 3 runs per side) | pytest-pr-1.log:77 «23 passed in 3.31s»

| P-C1 | On Python 3.14, ProxySchemeUnknown ends up with URLSchemeUnknown's prefix in front of its own

message. (PR description) | not observed | probe model-p1, c1.message: same on both sides: "Proxy URL had

unsupported scheme ftp, should use http:// or https://" | probe-model-p1-pr-1.jsonl:1 «"observation":

"c1.message"»

| P-C2 | The expected message (and what Python 3.13 prints) is Proxy URL had unsupported scheme ftp, should

use http:// or https://. (PR description) | proven | probe model-p1, c2.message_none: same on both sides: "Proxy

URL had no scheme, should start with http:// or https://" | probe-model-p1-pr-1.jsonl:2 «"observation":

"c2.message_none"»

| P-C3 | On Python 3.14, err.scheme is set to that whole message rather than "ftp". (PR description) | proven |

probe model-p1, c3.scheme: value: "AttributeError" -> "ftp" | probe-model-p1-pr-1.jsonl:3 «"observation":

"c3.scheme"»

| P-C4 | On Python 3.13 and older, err.scheme does not exist at all. (PR description) | false | probe model-p1,

c4.has_scheme_none: value: false -> true | probe-model-p1-pr-1.jsonl:4 «"observation":

"c4.has_scheme_none"»

| P-C5 | After the fix, ProxySchemeUnknown.__init__ calls LocationValueError.__init__ directly, making the result

independent of Python version and setting self.scheme appropriately (None for the no-scheme and localhost

cases). (PR description) | proven | probe model-p1, c5.is_locationvalueerror: same on both sides: true |

probe-model-p1-pr-1.jsonl:5 «"observation": "c5.is_locationvalueerror"»

| P-C7 | On Python 3.13 the same test fails with an AttributeError because ProxySchemeUnknown has no

scheme attribute. (PR description) | proven | probe model-p1, c7.scheme_eq_ftp: value: "AttributeError" -> true |

probe-model-p1-pr-1.jsonl:6 «"observation": "c7.scheme_eq_ftp"»

| P-C8 | After applying the fix, the test passes on both Python 3.13 and 3.14. (PR description) | not observed |

probe model-p1, c8.message_matches: same on both sides: true | probe-model-p1-pr-1.jsonl:7 «"observation":

"c8.message_matches"»

| P-C9 | After the fix, the full test suite passes on Python 3.13 and 3.14 and nox -s lint is clean. (PR description) |

proven | probe model-p1, c9.import_ok: same on both sides: true | probe-model-p1-pr-1.jsonl:8 «"observation":

"c9.import_ok"»

Citation check (PASS): 9 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 20 / 23 | | Tests unchanged | 20 | | New tests that pass | 3 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 37c5ff3f318a at

2026-11-09T06:21:43.172755+00:00): model-written probe of the changed files (no description given)

• P-C1 (PR description): On Python 3.14, ProxySchemeUnknown ends up with URLSchemeUnknown's prefix in

front of its own message. Expects the covered observations to differ.

• P-C2 (PR description): The expected message (and what Python 3.13 prints) is Proxy URL had unsupported

scheme ftp, should use http:// or https://. Expects the covered observations to stay the same.

• P-C3 (PR description): On Python 3.14, err.scheme is set to that whole message rather than "ftp". Expects the

covered observations to differ.

• P-C4 (PR description): On Python 3.13 and older, err.scheme does not exist at all. Expects the covered

observations to stay the same.

• P-C5 (PR description): After the fix, ProxySchemeUnknown.__init__ calls LocationValueError.__init__ directly,

making the result independent of Python version and setting self.scheme appropriately (None for the no-scheme

and localhost cases). Expects the covered observations to stay the same.

• P-C7 (PR description): On Python 3.13 the same test fails with an AttributeError because

ProxySchemeUnknown has no scheme attribute. Expects the covered observations to differ.

• P-C8 (PR description): After applying the fix, the test passes on both Python 3.13 and 3.14. Expects the

covered observations to differ.

• P-C9 (PR description): After the fix, the full test suite passes on Python 3.13 and 3.14 and nox -s lint is clean.

Expects the covered observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.message | P-C1 | no change although the claim says it changes | same on both sides: "Proxy URL had

unsupported scheme ftp, should use http:// or https://" | probe-model-p1-main-1.jsonl:1 «"observation":

"c1.message"»
probe-model-p1-pr-1.jsonl:1 «"observation": "c1.message"»

| c2.message_none | P-C2 | as claimed | same on both sides: "Proxy URL had no scheme, should start with

http:// or https://" | probe-model-p1-main-1.jsonl:2 «"observation":

"c2.message_none"»
probe-model-p1-pr-1.jsonl:2 «"observation": "c2.message_none"»

| c3.scheme | P-C3 | as claimed | value: "AttributeError" -> "ftp" | probe-model-p1-main-1.jsonl:3 «"observation":

"c3.scheme"»
probe-model-p1-pr-1.jsonl:3 «"observation": "c3.scheme"»

| c4.has_scheme_none | P-C4 | contradicts the claim | value: false -> true | probe-model-p1-main-1.jsonl:4

«"observation": "c4.has_scheme_none"»
probe-model-p1-pr-1.jsonl:4 «"observation":

"c4.has_scheme_none"»

| c5.is_locationvalueerror | P-C5 | as claimed | same on both sides: true | probe-model-p1-main-1.jsonl:5

«"observation": "c5.is_locationvalueerror"»
probe-model-p1-pr-1.jsonl:5 «"observation":

"c5.is_locationvalueerror"»

| c7.scheme_eq_ftp | P-C7 | as claimed | value: "AttributeError" -> true | probe-model-p1-main-1.jsonl:6

«"observation": "c7.scheme_eq_ftp"»
probe-model-p1-pr-1.jsonl:6 «"observation": "c7.scheme_eq_ftp"»

| c8.message_matches | P-C8 | no change although the claim says it changes | same on both sides: true |

probe-model-p1-main-1.jsonl:7 «"observation": "c8.message_matches"»
probe-model-p1-pr-1.jsonl:7

«"observation": "c8.message_matches"»

| c9.import_ok | P-C9 | as claimed | same on both sides: true | probe-model-p1-main-1.jsonl:8 «"observation":

"c9.import_ok"»
probe-model-p1-pr-1.jsonl:8 «"observation": "c9.import_ok"»

Citation check (PASS): 16 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (7.1 s run).

Coverage gate (python, confidence normal): 2 of 2 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| src/urllib3/exceptions.py | 2 | 2

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 1d6b2f1cf9cd of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 2 of 2 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 9 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 16 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | probe: 1 observation(s) need attention: c4.has_scheme_none differs

although the claim says it does not (C4); probe: 2 observation(s) shown as advisory only (claim not shown on

these inputs; a related change was proven): c1.message (C1); c8.message_matches (C8); probe: claim(s) C2,

C3, C5, C7, C9 proven; tests: 20 test(s) ran on main, 23 on the PR; 3 new test(s) pass, 20 unchanged | This is

about the PR, not the run: read the failing tests and their log lines and the Probes section, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 2 of 2 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 9 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 16 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).
• I recorded my decision (see below).
Decision recorded: none yet. To record one, run make decide
REPORT=poc-main/reports/daily/2026-11-09/urllib3-5368 DECISION=approve (or reject, approve_with_notes, re_run).

Rerun command
make check PR=https://github.com/urllib3/urllib3/pull/5368

Evidence links
• Pull request urllib3/urllib3#5368
• record.json
• pr.diff
• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-p1-main-1.jsonl
• probe-model-p1-pr-1.jsonl
• main run 1 result: pytest-main-1.log:77 «20 passed in 2.77s»
• pr run 1 result: pytest-pr-1.log:77 «23 passed in 3.31s»

Glossary
| Term | Meaning
| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.
| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 18:39 IST